

学号：202300130183	姓名：宋浩宇
实验名称：命名实体识别基础实验——中文实体识别与标注	
1. 实验目的	
1. 理解命名实体识别（NER）的基本概念。 2. 掌握常见实体类别（人名、地名、机构名等）。 3. 学习使用 Python 工具进行实体识别。 4. 理解实体识别在信息抽取中的作用。 5. 分析实体识别中的错误与歧义问题。	
2. 实验原理	
命名实体识别（Named Entity Recognition, NER）是自然语言处理中的基础任务，其目标是从文本中识别具有特定意义的实体，并为其赋予类别标签。 例如： 张三 在 北京大学 学习。 可以识别为： 张三 / PERSON 北京大学 / ORGANIZATION 常见实体类别包括： PERSON（人名） LOCATION（地名） ORGANIZATION（机构名） TIME（时间） NER 通常作为信息抽取的第一步，为后续关系抽取、事件抽取提供基础。	
3. 实验步骤与代码	
实验步骤： 1. 安装并导入 NER 工具库 2. 输入实验文本 3. 调用实体识别接口 4. 输出识别结果（实体 + 类型） 5. 对多个句子进行测试 6. 记录错误识别或漏识别情况 7. 分析结果	
代码： <pre>from __future__ import annotations import os import re from pathlib import Path # huggingface_hub 会在 import 阶段读取;XET 后端在部分网络下会长时间无进度, 看似卡死。 if "HF_HUB_DISABLE_XET" not in os.environ: os.environ["HF_HUB_DISABLE_XET"] = "1" os.environ.setdefault("HF_HUB_DOWNLOAD_TIMEOUT", "600")</pre>	

```

DATA_PATH = r"./exp-7.1-data.txt"
HF_NER_MODEL = os.environ.get(
    "NLP_WEEK7_NER_MODEL",
    "uer/roberta-base-finetuned-cluener2020-chinese",
)
def resolve_hf_token() -> tuple[str | None, str]:
    """
    返回 (token, 来源说明)。不把 token 写入日志或打印明文。
    在首次 import transformers / huggingface_hub 之前调用, 并写入
    os.environ["HF_TOKEN"] 供 Hub 使用。
    """
    for key in ("HF_TOKEN", "HUGGING_FACE_HUB_TOKEN"):
        raw = os.environ.get(key)
        if raw and raw.strip():
            return raw.strip(), f"环境变量 {key}"
    path = os.environ.get("NLP_WEEK7_HF_TOKEN_FILE", "").strip()
    if path:
        p = Path(path).expanduser()
        if not p.is_file():
            return None, f"(NLP_WEEK7_HF_TOKEN_FILE 指向的文件不存在: {path})"
        for line in p.read_text(encoding="utf-8").splitlines():
            s = line.split("#", 1)[0].strip()
            if s:
                return s, f"文件 {p}"
        return None, f"(NLP_WEEK7_HF_TOKEN_FILE 文件无有效行: {p})"
    return None, "(未配置, 将匿名访问 Hub)"
def apply_hf_token_before_hub_import() -> tuple[str | None, str]:
    """把令牌同步到进程环境, 便于 huggingface_hub 在 import 后仍使用同一令牌。"""
    token, src = resolve_hf_token()
    if token:
        os.environ["HF_TOKEN"] = token
        return token, src
def read_sentences(path: str) -> list[str]:
    lines = []
    for raw in Path(path).read_text(encoding="utf-8").splitlines():
        s = raw.strip()
        if s:
            lines.append(s)
    return lines
def map_msra_tag(tag: str) -> str:

```

```

t = (tag or "").upper()
if t in ("NR", "PER", "PERSON"):
    return "PERSON"
if t in ("NS", "LOC", "LOCATION", "GPE"):
    return "LOCATION"
if t in ("NT", "ORG", "ORGANIZATION"):
    return "ORGANIZATION"
if t in ("T", "TIME"):
    return "TIME"
return tag or "MISC"
def map_clue_group(group: str) -> str:
    g = (group or "").lower()
    if g == "name":
        return "PERSON"
    if g in ("address",):
        return "LOCATION"
    if g in ("organization", "company", "government"):
        return "ORGANIZATION"
    if g in ("time",):
        return "TIME"
    return group or "MISC"
def _flatten_hanlp_ner_item(item, out: list[tuple[str, str, int | None, int | None]]) -> None:
    if item is None:
        return
    if isinstance(item, (list, tuple)):
        if len(item) >= 2 and isinstance(item[0], str) and isinstance(item[1], str):
            ent, tag = item[0], item[1]
            s = int(item[2]) if len(item) > 2 and item[2] is not None else None
            e = int(item[3]) if len(item) > 3 and item[3] is not None else None
            out.append((ent, tag, s, e))
            return
        for sub in item:
            _flatten_hanlp_ner_item(sub, out)
        return
    if isinstance(item, dict):
        for v in item.values():
            _flatten_hanlp_ner_item(v, out)
def normalize_hanlp_output(raw, n_sentences: int) -> list[list[tuple[str, str, int | None, int | None]]]:
    if raw is None:

```

```

        return [[] for _ in range(n_sentences)]
    per: list[list[tuple[str, str, int | None, int | None]]] = []
    def is_span_list(obj) -> bool:
        if not isinstance(obj, list) or not obj:
            return False
        first = obj[0]
        return (
            isinstance(first, (list, tuple))
            and len(first) >= 2
            and isinstance(first[0], str)
            and isinstance(first[1], str)
        )
    if n_sentences == 1:
        if is_span_list(raw):
            spans: list[tuple[str, str, int | None, int | None]] =
[]
            _flatten_hanlp_ner_item(raw, spans)
            return [spans]
        if isinstance(raw, list) and raw and isinstance(raw[0],
list):
            for sent in raw:
                spans = []
                _flatten_hanlp_ner_item(sent, spans)
                per.append(spans)
            if len(per) == 1:
                return per
            if len(per) == 0:
                return [[]]
            spans = []
            _flatten_hanlp_ner_item(raw, spans)
            return [spans]
        if isinstance(raw, list):
            for sent in raw:
                spans = []
                _flatten_hanlp_ner_item(sent, spans)
                per.append(spans)
            if per:
                if len(per) > n_sentences:
                    return per[:n_sentences]
                if len(per) < n_sentences:
                    return per + [[] for _ in range(n_sentences -
len(per))]
            return per
        spans = []

```

```

_flatten_hanlp_ner_item(raw, spans)
if n_sentences <= 1:
    return [spans]
return [spans] + [[] for _ in range(n_sentences - 1)]
def try_build_hanlp_ner():
    import hanlp
    name = "MSRA_NER_BERT_BASE_ZH"
    if not hasattr(hanlp.pretrained.ner, name):
        for cand in ("MSRA_NER_ALBERT_BASE_ZH",
"MSRA_NER_ELECTRA_SMALL_ZH"):
            if hasattr(hanlp.pretrained.ner, cand):
                name = cand
                break
    path = getattr(hanlp.pretrained.ner, name)
    return hanlp.load(path), f"hanlp:{name}"
def try_build_hf_ner():
    print(
        "\n[加载] 正在初始化 Transformers NER(首次会从 Hub 下载权
重, 体积较大, 终端可能数分钟无新输出, 属正常现象)。",
        flush=True,
    )
    hf_token, hf_src = apply_hf_token_before_hub_import()
    use_cli_token = _env_truthy("NLP_WEEK7_USE_HF_CLI_TOKEN")
    if hf_token:
        print(f"[加载] Hub 鉴权: 已提供访问令牌(来源: {hf_src}), 长
度 {len(hf_token)}。", flush=True)
    elif use_cli_token:
        print("[加载] Hub 鉴权: NLP_WEEK7_USE_HF_CLI_TOKEN=1, 将使
用 huggingface-cli 已登录缓存(token=True)。", flush=True)
    else:
        print(f"[加载] Hub 鉴权: {hf_src}", flush=True)
    print(
        "[加载] HF_HUB_DISABLE_XET=%s HF_ENDPOINT=%s"
        % (
            os.environ.get("HF_HUB_DISABLE_XET", ""),
            os.environ.get("HF_ENDPOINT", "") or "(未设置)",
        ),
        flush=True,
    )
    from transformers import pipeline
    device = -1
    torch_ver = ""
    try:
        import torch

```

```

    torch_ver = getattr(torch, "__version__", "")
    device = 0 if torch.cuda.is_available() else -1
except Exception:
    device = -1
print("[加载] 开始构建 pipeline(下载结束后会打印推理结果)...",
flush=True)
pipe_kw: dict = {
    "model": HF_NER_MODEL,
    "tokenizer": HF_NER_MODEL,
    "aggregation_strategy": "simple",
    "device": device,
    "trust_remote_code": True,
}
if hf_token:
    pipe_kw["token"] = hf_token
elif use_cli_token:
    pipe_kw["token"] = True
clf = pipeline("ner", **pipe_kw)
dev_s = "cuda" if device == 0 else "cpu"
auth = "token" if hf_token else ("cli" if use_cli_token else
"anon")
meta =
f"transformers+pytorch:{HF_NER_MODEL} device={dev_s} hub_auth={
auth}"
if torch_ver:
    meta += f" torch={torch_ver}"
return clf, meta
def run_hanlp_on_sentences(ner, sentences: list[str]) ->
list[list[tuple[str, str, int | None, int | None]]]:
    if len(sentences) == 1:
        raw = ner(sentences[0])
        return normalize_hanlp_output(raw, 1)
    raw = ner(sentences)
    return normalize_hanlp_output(raw, len(sentences))
def run_hf_on_sentences(ner, sentences: list[str]) ->
list[list[tuple[str, str, int | None, int | None]]]:
    out: list[list[tuple[str, str, int | None, int | None]]] = []
    for s in sentences:
        rows = ner(s)
        spans: list[tuple[str, str, int | None, int | None]] = []
        for ent in rows or []:
            word = ent.get("word", "")
            grp = ent.get("entity_group", ent.get("label", ""))
            spans.append((word, str(grp), int(ent.get("start",

```

```

-1)), int(ent.get("end", -1)))
    out.append(spans)
    return out
def print_table(title: str, backend: str, sentences: list[str],
per_sent_spans: list[list[tuple[str, str, int | None, int | None]]],
mapper) -> None:
    print(f"\n--- {title} ---")
    print(f"后端: {backend}")
    print("序号 | 原句 | 实体 | 原始标签 | 归一类型 | 字符起止(若有)")
    print("-" * 72)
    for i, (sent, spans) in enumerate(zip(sentences,
per_sent_spans), 1):
        if not spans:
            print(f"{i:>2} | {sent} | (无实体) | | | ")
            continue
        for j, (text, raw_tag, s, e) in enumerate(spans):
            norm = mapper(raw_tag)
            se = "" if s is None or e is None or s < 0 else f"{s}-{e}"
            prefix = f"{i:>2} | {sent}" if j == 0 else f"{'':>2} |
{'':<}"
            print(f"{prefix} | {text} | {raw_tag} | {norm} | {se}")
def extract_times_regex(sentence: str) -> list[str]:
    return re.findall(r"\d{4}\s*年\s*\d{1,2}\s*月\s*\d{1,2}\s*日", sentence)
def _env_truthy(name: str) -> bool:
    v = os.environ.get(name, "").strip().lower()
    return v in ("1", "true", "yes", "on")
def main():
    print(f"数据文件: {DATA_PATH}")
    print(f"NER 模型: {HF_NER_MODEL}")
    print("默认优先 PyTorch 路线(Transformers)。若需先 HanLP, 请设置 NLP_WEEK7_TRY_HANLP_FIRST=1。")
    print("若首次下载模型较慢, 请配置代理或 HF_ENDPOINT 等镜像环境变量。")
    print("Hub 令牌: 可设 HF_TOKEN、HUGGING_FACE_HUB_TOKEN, 或 NLP_WEEK7_HF_TOKEN_FILE 指向单行密钥文件。")
    print(
        "Hub 提示: 默认 HF_HUB_DISABLE_XET=1 以避免 XET 下载路径卡住;若你已在环境显式设为 0, 将按你的设置使用。"
    )
    sentences = read_sentences(DATA_PATH)
    if not sentences:
        print("数据文件为空, 请检查路径与内容。")

```

```

        return
    print("\n--- 实验十三 命名实体识别:逐句结果(便于报告截图)
    ---")
    hanlp_first = _env_truthy("NLP_WEEK7_TRY_HANLP_FIRST")
    order: list[tuple[str, object, object, object, str]] = []
    if hanlp_first:
        order = [
            ("hanlp", try_build_hanlp_ner, run_hanlp_on_sentences,
            map_msra_tag, "HanLP(任务书可选工具)"),
            ("hf", try_build_hf_ner, run_hf_on_sentences,
            map_clue_group, "Transformers 与 PyTorch(默认推荐)"),
        ]
    else:
        order = [
            ("hf", try_build_hf_ner, run_hf_on_sentences,
            map_clue_group, "Transformers 与 PyTorch(默认推荐)"),
            ("hanlp", try_build_hanlp_ner, run_hanlp_on_sentences,
            map_msra_tag, "HanLP(可选)"),
        ]
    last_err: Exception | None = None
    for key, build, run, mapper, title in order:
        try:
            ner, backend = build()
            spans = run(ner, sentences)
            print_table(title, backend, sentences, spans, mapper)
            last_err = None
            break
        except Exception as exc:
            last_err = exc
            print(f"\n{title.split('(')[0]} 未启用或失败, 尝试下一
            后端。")
            print(f" 异常类型: {type(exc).__name__} 信息:
            {exc}")
            low = str(exc).lower()
            if key == "hanlp" and (
                "tensorflow" in low or "hanlp[full]" in low or
                "required by this model" in low
            ):
                print(' 提示: HanLP 该模型需 pip install
                "hanlp[full]" -U, 或改用默认 PyTorch 路线(勿设
                NLP_WEEK7_TRY_HANLP_FIRST)。')
            if key == "hf" and ("28" in str(exc) or "no space" in low):
                print(" 提示: 若为磁盘空间不足, 请清理 C 盘或设置
                TEMP、TMP、PIP_CACHE_DIR 到大空间盘后再下载模型。")

```



```

else:
    if last_err is not None:
        print("\n 所有后端均失败。PyTorch 路线请确认: pip
install torch transformers huggingface_hub, 并保证可联网或代理下载
模型。")
    return
print("\n--- 时间表达式(正则辅助, 可与模型 TIME 类型对照) ---")
for i, s in enumerate(sentences, 1):
    ts = extract_times_regex(s)
    print(f" [{i}] {s}")
    print(f"      正则抽取到的日期片段: {ts if ts else '(无)'}")
if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print(
            "\n[中断] 已取消模型下载或加载。若曾长时间无输出, 多为网
络或 Hub 存储后端导致, "
            "请保持代理开启, 或设置 HF_ENDPOINT 镜像, 并确认
HF_HUB_DISABLE_XET=1 后重试。",
            flush=True,
        )
        raise SystemExit(130) from None

```

4. 实验结果（图表与输出）

输出结果为：

F:\Homework\自然语言处理作业\venv\Scripts\python.exe F:\Homework\自然语言处理作业
\第七周实验\实验十三-命名实体识别实验——信息抽取基础.py

数据文件: ./exp-7.1-data.txt

NER 模型: uer/roberta-base-finetuned-cluener2020-chinese

默认优先 PyTorch 路线（Transformers）。若需先 HanLP，请设置
NLP_WEEK7_TRY_HANLP_FIRST=1。

若首次下载模型较慢，请配置代理或 HF_ENDPOINT 等镜像环境变量。

Hub 令牌：可设 HF_TOKEN、HUGGING_FACE_HUB_TOKEN，或
NLP_WEEK7_HF_TOKEN_FILE 指向单行密钥文件。

Hub 提示：默认 HF_HUB_DISABLE_XET=1 以避免 XET 下载路径卡住；若你已在环
境显式设为 0，将按你的设置使用。

--- 实验十三 命名实体识别：逐句结果（便于报告截图） ---

[加载] 正在初始化 Transformers NER（首次会从 Hub 下载权重，体积较大，终端可能数
分钟无新输出，属正常现象）。

[加载] Hub 鉴权：已提供访问令牌（来源：环境变量 HF_TOKEN），长度 37。

[加载] HF_HUB_DISABLE_XET=1 HF_ENDPOINT=（未设置）

F:\Homework\自然语言处理作业\.venv\lib\site-packages\torch\cuda_init__.py:61: FutureWarning: The pynvml package is deprecated. Please install nvidia-ml-py instead. If you did not install pynvml directly, please report this to the maintainers of the package that installed pynvml for you.

```
import pynvml # type: ignore[import]
```

[加载] 开始构建 pipeline (下载结束后会打印推理结果) ...

F:\Homework\自然语言处理作业\.venv\lib\site-packages\huggingface_hub\file_download.py:138: UserWarning:

'huggingface_hub' cache-system uses symlinks by default to efficiently store duplicated files but your machine does not support them in C:\Users\23676\cache\huggingface\hub\models--uer--roberta-base-finetuned-cluener2020-chinese. Caching files will still work but in a degraded version that might require more space on your disk. This warning can be disabled by setting the 'HF_HUB_DISABLE_SYMLINKS_WARNING' environment variable. For more details, see https://huggingface.co/docs/huggingface_hub/how-to-cache#limitations.

To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate developer mode, see this article: <https://docs.microsoft.com/en-us/windows/apps/get-started/enable-your-device-for-development>

```
warnings.warn(message)
```

Loading weights: 100%[████████████████████] 199/199 [00:00<00:00, 21818.49it/s]

--- Transformers 与 PyTorch (默认推荐) ---

后端: transformers+pytorch:uer/roberta-base-finetuned-cluener2020-chinese device=cuda
hub_auth=token torch=2.6.0+cu124

序号 | 原句 | 实体 | 原始标签 | 归一类型 | 字符起止(若有)

-
- | | | | | | |
|---|-----------------------|--------------|--------------|--------------|-------|
| 1 | 张三在北京大学学习。 | 张三 | name | PERSON | 0-2 |
| | 北京大学 | organization | ORGANIZATION | 3-7 | |
| 2 | 李雷来自山东济南。 | 李雷 | name | PERSON | 0-2 |
| | 山东济南 | address | LOCATION | 4-8 | |
| 3 | 华为公司发布了新产品。 | 华为公司 | company | ORGANIZATION | 0-4 |
| 4 | 2025年3月15日, 王老师在上海开会。 | 老师 | position | position | 12-14 |
| | 上海 | address | LOCATION | 15-17 | |
| 5 | OpenAI 发布了新模型。 | (无实体) | | | |

--- 时间表达式 (正则辅助, 可与模型 TIME 类型对照) ---

- [1] 张三在北京大学学习。
正则抽取到的日期片段: (无)
- [2] 李雷来自山东济南。
正则抽取到的日期片段: (无)
- [3] 华为公司发布了新产品。
正则抽取到的日期片段: (无)
- [4] 2025年3月15日, 王老师在上海开会。

正则抽取到的日期片段: ['2025 年 3 月 15 日'] [5] OpenAI 发布了新模型。 正则抽取到的日期片段: (无)
--

图表结果为:

无图表结果

5. 结果分析

1.本实验中识别出了哪些类型的实体？

答：从控制台可见，模型给出的原始标签主要包括 name、organization、address、company、position 等。脚本将其中一部分映射为任务书常见的粗粒度类型，例如 name 映射为 PERSON，address 映射为 LOCATION，organization 与 company 映射为 ORGANIZATION。另有 position 在输出中仍以 position 形式出现，表示职位类槽位，与典型人名标签不同。正则辅助输出中还单独列出了日期串 2025 年 3 月 15 日，可与模型是否标 TIME 对照，但本次表格中未出现 TIME 类实体。

2.哪些句子的识别结果较准确？

答：第 1 句与第 2 句整体较可用。第 1 句识别出与人名、机构相关的跨度，第 2 句识别出人名与地址类跨度，和句子语义大体一致。第 3 句识别出公司类跨度，与“华为公司”这一机构叙述方向一致。以上几句可作为“相对较准确”的示例写入报告，但仍可讨论字间空格等展示问题。

3.哪些实体识别存在错误或遗漏？

答：第 4 句问题较明显。人名“王老师”在输出里只出现“老师”一段，且类型为 position，字符区间约为 12 到 14，属于对人名边界的截断与类型误判。同句中“上海”被标为 address，地点要素有一定覆盖，但与人名错误相比，整句人物信息不完整。第 5 句对“OpenAI”未给出任何实体，属于对英文机构名的漏检。另外，多句实体文本在展示上带空格，例如“张三”“北京大学”，与连续汉字书写不一致，这不一定是标签全错，但会影响阅读与后续对齐，可在报告中记为展示层面的缺陷。

4.中文实体与英文实体（如 OpenAI）识别效果有何差异？

答：本次输出中，中文人名、地名、公司名等多数能给出非空标注，而英文机构名 OpenAI 所在句子为无实体输出，呈现明显差异。原因通常包括训练语料以中文为主、子词切分与标签体系对英文专名覆盖不足，以及句子较短时边界更难稳定等。实验报告可结合本句写成“英文专名漏检”的对照案例。

5.为什么命名实体识别是信息抽取的基础？

答：信息抽取往往要把非结构化文本变成可计算的结构，第一步需要知道哪些片段是“谁、哪里、哪个组织、什么时间”等关键对象。命名实体识别把这些对象从文本中定位并分类，为后续关系抽取、事件抽取、问答与知识库对齐提供锚点。没有可靠的实体边界与类型，后

续槽位填充和关系论证容易错位或无法启动，因此 NER 常被视为信息抽取链路的前置基础环节。

6. 实验总结

本次在配置好网络与 Hub 令牌后，使用 Transformers 与 PyTorch 加载 uer/roberta-base-finetuned-cluener2020-chinese，对 exp-7.1-data.txt 中各句完成命名实体识别，控制台按句输出了实体、原始标签、归一类型与字符区间，并附日期正则对照。程序正常结束，输出可用于截图写入报告，同时留下了若干错例与英文漏检可供分析。

使用 LLM 辅助完成代码编写